



Boundary Value Analysis & Equivalence Partitioning

Finding Bugs at the Edge

Tags: QA, Testing, Test Design

Reading Time: 3 min

TL;DR

Equivalence Partitioning reduces the number of tests (grouping similar inputs).

Boundary Value Analysis finds the most bugs (testing the edges of those groups).

The Core Concepts

We cannot test every single possible input (e.g., numbers 1 to 1,000,000). That is impossible. We use these two techniques to test **smarter**, not harder.

1. Equivalence Partitioning (The Grouping)

Goal: To test less by grouping similar items.

Instead of testing every number, we group them. If inputs 1 to 100 are valid, we don't need to test 5, 6, and 7. They are "Equivalent."

- If 5 works, 6 will probably work too. We just pick one.

🚧 2. Boundary Value Analysis (The Edges)

Goal: To catch bugs where they hide most often.

Developers usually make mistakes at the "boundaries" of logic. We test the exact edges of the groups.

- If the range is 1-100, we test: 0, 1, 100, 101.
-

🎮 Real World Example: The Health Bar

Imagine a game character can have Health between **0** and **100**.

Using Equivalence Partitioning:

We pick one value from each "zone."

- Valid Zone (0-100): Test **50**.
- Invalid Zone (Below 0): Test **10**.
- Invalid Zone (Above 100): Test **150**.

Using Boundary Value Analysis:

We test the specific lines between zones.

- Lower Boundary: Test **1** (Invalid) and **0** (Valid/Dead).
 - Upper Boundary: Test **100** (Valid/Max) and **101** (Invalid).
-

🧠 Why This Matters (The "Off-By-One" Error)

This is where coding knowledge helps a tester. In my C# studies, I learned how easy it is for a developer to write:

```
if (health < 0)
```

Instead of:

```
if (health <= 0)
```

Boundary Value Analysis catches this specific error. Testing the number "50" will never catch it, but testing "0" will.